Fuente partida regulable con control FreeRTOS e interfaz Linux

Adjustable dual-rail power supply with FreeRTOS control and Linux interface

Brian Benavidez

Universidad Tecnológica Nacional, Facultad Regional Avellaneda, Avellaneda, Buenos Aires, Argentina

Resumen

Se presenta el diseño y la validación de una fuente partida de dos ramas regulables, controlada por una Raspberry Pi Pico 2 W con FreeRTOS e integrada con una Raspberry Pi Zero 2 W. Cada salida emplea una etapa flyback no aislada, regulación PI, arranque suave y protecciones digitales de sobrecorriente y sobretensión. Las tensiones se adquieren mediante un ADS1115 y las corrientes con el ADC interno. La ampliación EGB incorpora un controlador Linux basado en serdev, un dispositivo de carácter, una aplicación de consola y un panel web. La comunicación utiliza UART a 115200 bit/s y un protocolo textual con respuestas OK o ERR. La validación individual se realizó con cargas cementadas cercanas a 22 Ω y un multímetro verdadero valor eficaz GRALF GMF-39D. Además, se ensayaron ambas ramas simultáneamente a 5,00 V con cargas de 22,0 Ω y 22,2 Ω: la entrada fue de 10,00 V, 0,427 A y 4,275 W, y el rendimiento total calculado fue aproximadamente 53 %. El error de tensión permaneció por debajo de 0,10 V y el error de corriente por debajo de 10 mA. También se comprobaron el enclavamiento y el restablecimiento de las protecciones. La versión final de la Pico identifica las ramas como 0 y 1; los clientes Linux conservan su interfaz P y N y traducen esos nombres en el límite UART. La recepción UART usa una interrupción y una cola de bytes independiente; mediciones y comandos UART/encoder convergen en una única cola central tipada. Esta versión se cargó en la placa y se verificó desde el panel web mediante tres lecturas GET completas con las salidas apagadas.

Palabras clave: fuente partida, FreeRTOS, Linux embebido, UART, controlador de dispositivo.

Abstract

This paper presents the design and validation of a two-channel adjustable dual-rail power supply controlled by a Raspberry Pi Pico 2 W running FreeRTOS and integrated with a Raspberry Pi Zero 2 W. Each output uses a non-isolated flyback stage, PI regulation, soft start, and digital overcurrent and overvoltage protection. Output voltages are acquired by an ADS1115, while currents are sampled by the internal ADC. The EGB extension adds a Linux serdev driver, a character device, a command-line application, and a web dashboard. Communication uses a 115200 bit/s UART link and a text protocol with OK or ERR responses. Individual-rail validation used approximately 22 Ω cement resistors and a GRALF GMF-39D true-RMS multimeter. An additional simultaneous test operated both rails at 5.00 V with 22.0 Ω and 22.2 Ω loads; the 10.00 V input provided 0.427 A and 4.275 W, yielding an estimated total efficiency of about 53 %. Voltage error remained below 0.10 V and current error below 10 mA. Protection latching and manual reset were also verified. The final Pico firmware identifies the rails as 0 and 1, while the Linux clients retain their P and N user interface and translate those names at the UART boundary. UART reception uses an interrupt and a separate byte queue; measurements and UART/encoder commands converge into one typed central queue. This version was flashed to the board and verified from the web dashboard through three complete GET reads with both outputs disabled.

Keywords: dual-rail power supply, FreeRTOS, embedded Linux, UART, device driver.

1. Introducción

Las fuentes de laboratorio requieren regulación estable, medición comprensible y una respuesta segura ante fallas. En este trabajo se integraron dos etapas complementarias. La EGA resuelve la adquisición, el control en tiempo real, la interfaz local y las protecciones sobre una Raspberry Pi Pico 2 W. La EGB incorpora una Raspberry Pi Zero 2 W con Linux para exponer el equipo mediante consola y navegador, sin reemplazar las funciones críticas del microcontrolador.

La separación responde a los requisitos temporales de cada dominio. FreeRTOS planifica tareas periódicas de medición y control, mientras Linux aporta el modelo de controladores, el acceso mediante archivos y los servicios de usuario [1]–[4]. El objetivo fue mantener una implementación sencilla de explicar y reproducir, con un protocolo UART legible y con límites validados tanto en el firmware como en las aplicaciones.

2. Desarrollo

2.1. Etapa de potencia, medición y control

La fuente posee dos ramas no aisladas con masa común. Los MOSFET FQP30N06L son excitados por un TC4427 y conmutan a 50 kHz. El ciclo útil se limita al 70 % para alcanzar el rango previsto sin llevar el convertidor fuera de la zona de ensayo. Cada rama implementa un arranque suave de 1 s y un regulador PI con Kp = 0,80 y Ki = 3,50. Los snubbers RCD reducen los picos de drenaje; durante los ensayos se observaron picos próximos a 40 V, por debajo del nivel de 47 V considerado en el diseño.

El ADS1115 digitaliza las tensiones de salida y el ADC interno mide las señales acondicionadas de corriente y la tensión de entrada. Las conversiones se calibran con las relaciones obtenidas experimentalmente. La tarea protection_task lee Vin, I- e I+ cada 1 ms; si una corriente alcanza su límite, enclava la falla, deshabilita la rama y lleva su PWM a cero. La sobretensión se comprueba durante la adquisición del ADS1115 y también mantiene la salida apagada hasta recibir RST.

Tabla 1. Parámetros principales de la EGA.

Parámetro	Valor
Referencia de salida	0 a 30 V por rama; paso de 0,5 V; valor inicial de 5 V
Límite de corriente	0,10 a 0,75 A; paso de 0,05 A
Límite de sobretensión	5 a 40 V; paso de 0,5 V
PWM	50 kHz; ciclo útil máximo de 70 %
Control	PI: Kp = 0,80; Ki = 3,50; arranque suave ≈ 1 s
Adquisición	ADS1115 para tensión; ADC interno para corriente y entrada

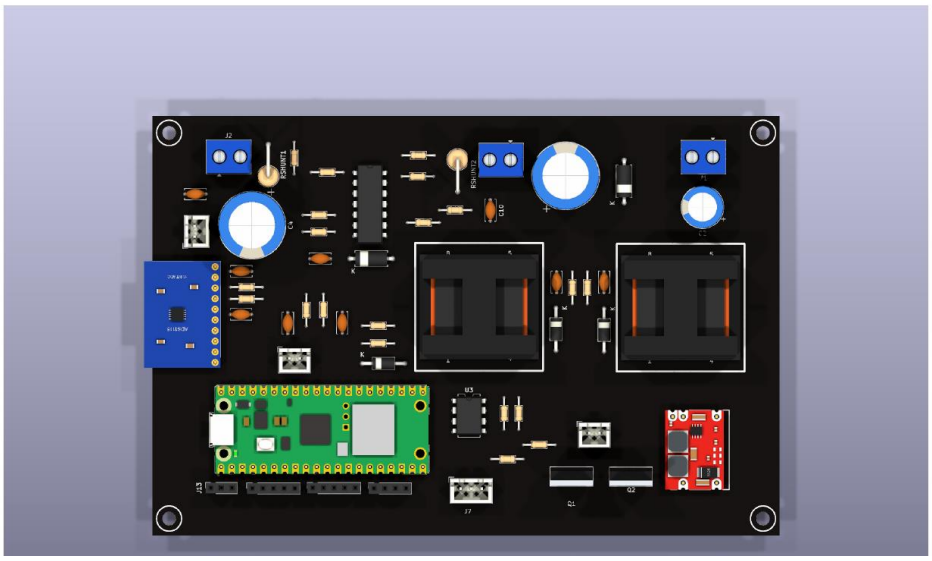


Figura 1. Vista tridimensional de la placa de circuito impreso desarrollada.

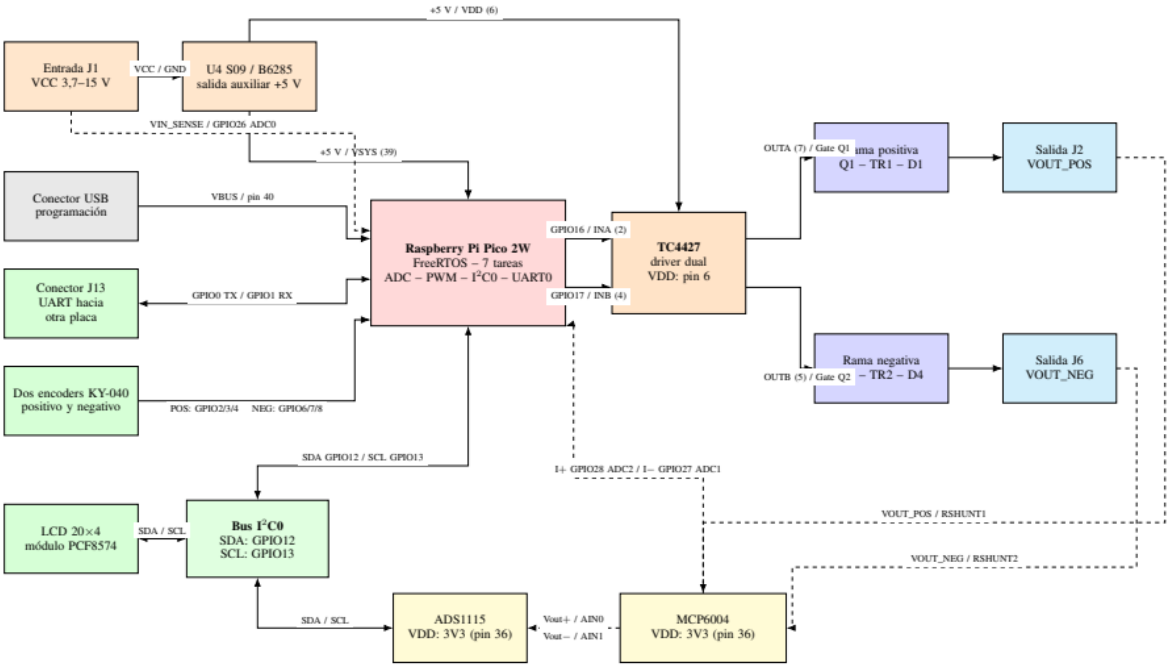


Figura 2. Conexión funcional de los módulos de la EGA.

2.2. Organización del firmware FreeRTOS

El firmware se organiza en diez tareas: protection_task, adc_internal_task, central_task, ads1115_task, control_task (+), control_task (-), encoder_task, uart_task, display_task y flash_task. Mediciones y comandos UART/encoder convergen como eventos tipados en central_event_queue; uart_rx_queue conserva los bytes de la interrupción UART y encoder_queue los eventos GPIO. central_task espera hasta 5 ms por el primer evento y procesa una ráfaga acotada antes de revisar la caducidad de 150 ms, aun con tráfico continuo. El mutex I2C se libera entre transacciones ADS1115

y entre escrituras del LCD. Las conversiones de ambas ramas son secuenciales y la pantalla se actualiza cada 200 ms. flash_task guarda consignas y límites. La versión final es 1_ega/2_workspace/firmware/firmware.c; 2_egb/2_rtos/firmware/firmware.c queda sólo como referencia histórica.

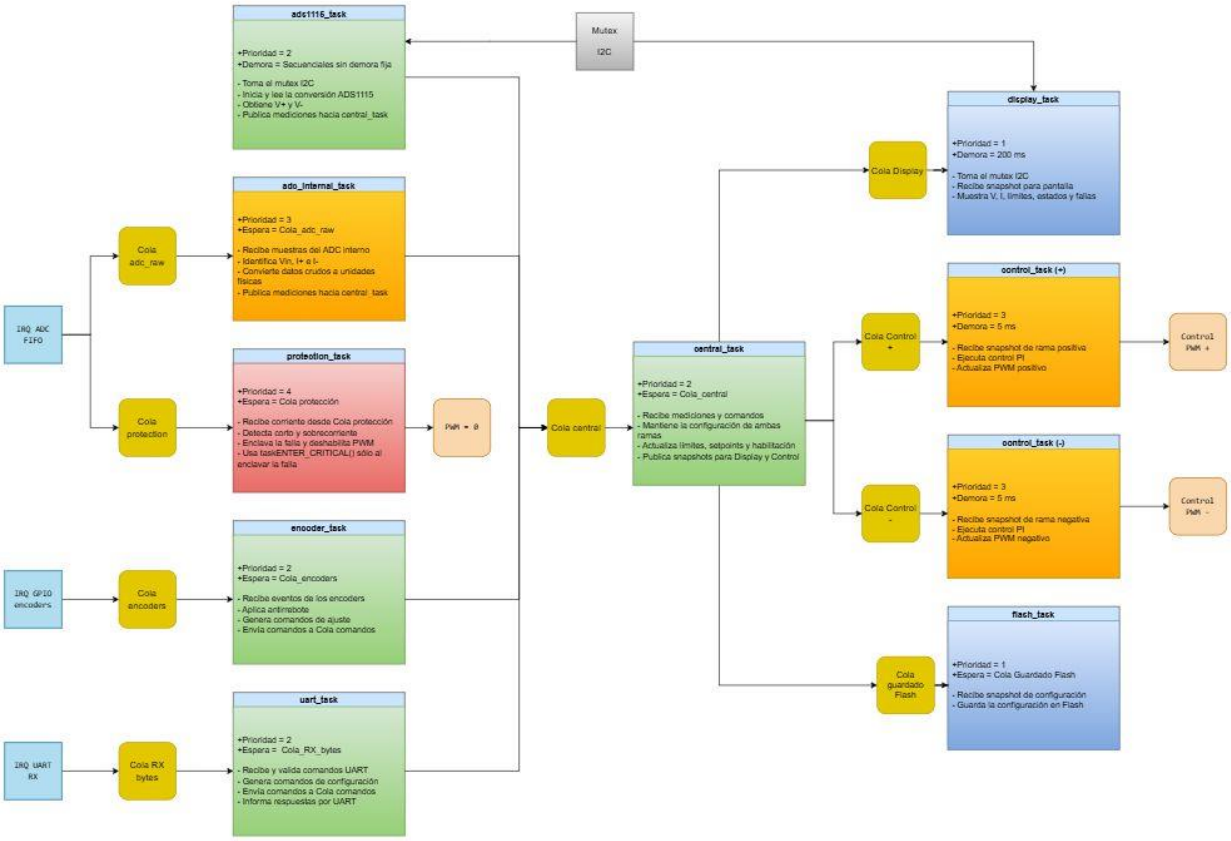


Figura 3. Arquitectura final del firmware de la Pico: interrupciones, cola RX UART, cola central unificada, tareas y recursos FreeRTOS.

2.3. Integración Linux de la EGB

La Raspberry Pi Zero 2 W se comunica con la Pico mediante UART a 115200 bit/s, formato 8N1 y niveles lógicos de 3,3 V. El enlace cruza TX y RX y comparte GND. Un overlay propio del árbol de dispositivos declara el periférico y lo asocia con un controlador serdev. El módulo crea /dev/ega_uart y ofrece open, read, write y poll. La recepción usa un FIFO protegido por spinlock y una cola de espera para las lecturas bloqueantes. El controlador serializa cada operación completa para impedir que dos aplicaciones repartan entre sí los bytes de una misma respuesta.

Sobre el dispositivo de carácter se implementaron ega-cli y un servicio web en el puerto 8080. El controlador serdev y /dev/ega_uart transportan texto sin interpretarlo. Ambos clientes conservan la interfaz P/N y traducen en el límite al protocolo final: P corresponde a 0, N a 1, SET se convierte en V, IL o VL y RESET en RST. Las confirmaciones OK QUEUED vuelven a mostrar el comando y el panel interpreta el GET ampliado. El parser conserva compatibilidad con PROT histórico, pero el firmware final informa las fallas mediante GET y no emite PROT espontáneo.

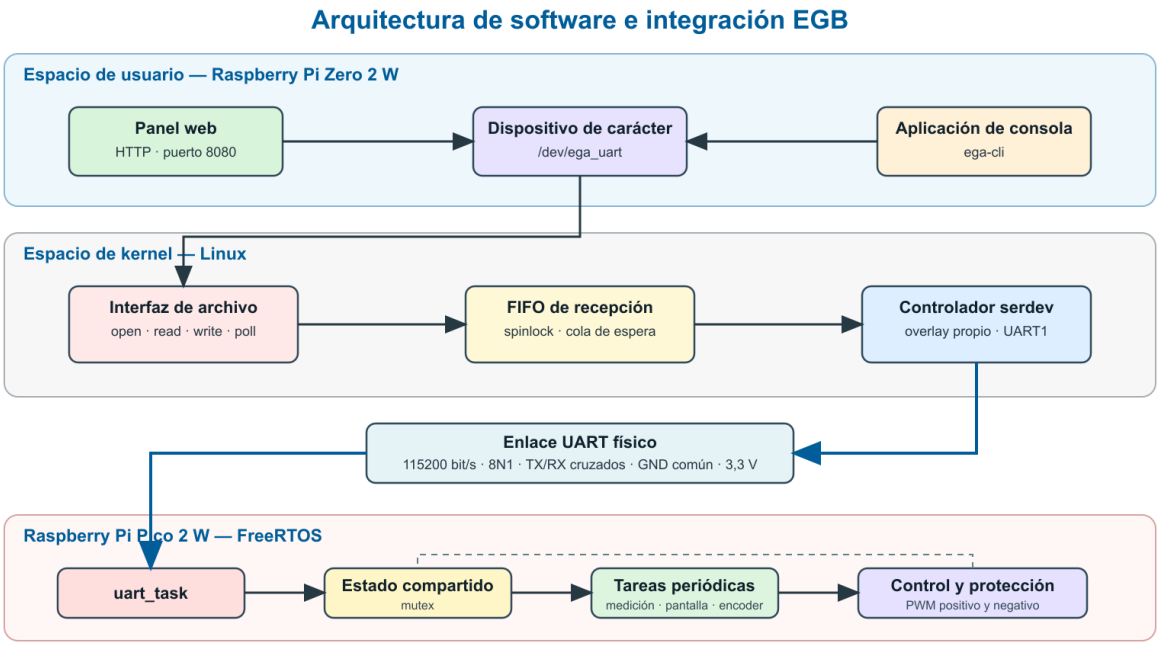


Figura 4. Integración EGB original; los clientes actuales conservan P/N y traducen al protocolo final 0/1 en el límite UART.

2.4. Protocolo y comandos

Cada comando ASCII termina con nueva línea y el retorno de carro se ignora. La ISR UART encola bytes en uart_rx_queue; uart_task espera sobre ella y arma la línea. Ante saturación descarta hasta recuperar una frontera válida. Las protecciones son independientes y sus enclavamientos se consultan con GET. Las ramas son 0 (positiva) y 1 (negativa). V, IL y VL cambian consignas y límites; OUT habilita o apaga, RST borra el enclavamiento sin reactivar, GET devuelve estado completo y SAVE guarda con ambas salidas apagadas. Los cambios aceptados ingresan en la cola central junto con mediciones y encoder; OK QUEUED se envía sólo al encolar y una cola llena produce ERR BUSY.

Tabla 2. Comandos aceptados por el firmware final de la Pico; 0 identifica la rama positiva y 1 la negativa.

Comando	Función
GET	Lee mediciones, consignas, límites, validez, estados, fallas y ciclo útil.
V <0|1> <V>	Modifica la referencia de la rama seleccionada.
IL <0|1> <A>	Modifica el límite de corriente de la rama seleccionada.
VL <0|1> <V>	Modifica el límite de sobretensión de la rama seleccionada.
OUT <0|1> ON|OFF	Habilita o deshabilita la rama seleccionada.
RST <0|1>	Borra OC/OV y mantiene la salida apagada.
SAVE	Guarda consignas y límites en la memoria flash.

2.5. Método de verificación

Las pruebas se realizaron primero por rama y luego con el sistema completo. Se emplearon resistencias cementadas de 5 W, con valores medidos de 22,2 Ω para la rama positiva y 22,0 Ω para la negativa. Las tensiones y las caídas sobre los shunts se contrastaron con un multímetro verdadero valor eficaz GRALF GMF-39D. La fuente de laboratorio proporcionó la tensión de entrada y permitió registrar corriente y potencia consumidas.

La calibración comparó el instrumento con la lectura digital y ajustó las tablas. Luego se ensayaron los umbrales OC/OV, el apagado PWM, el enclavamiento, RST y el retorno apagado. La validación EGB comprobó módulo, /dev/ega_uart, consola, panel y respuestas P/N. La traducción 0/1 se verificó con pruebas de mapeo, GET ampliado y un transporte simulado que incluyó PROT histórico; la consola compiló en la Zero, su prueba auxiliar y las 11 pruebas web finalizaron correctamente. Antes de la arquitectura final se obtuvieron tres GET por consola y tres por web, con respuestas habituales de 21 ms a 28 ms. El diagnóstico USB de 1 Hz registró Vin, salidas, habilitación, validez, edad, fallas y ciclo útil, sin sustituir una medición transitoria. Por último se compiló, enlazó y cargó la versión con IRQ UART directa y cola central unificada. Tres GET web recorrieron panel, /dev/ega_uart y UART GPIO y devolvieron HTTP 200 y OK completo: 590,525 ms el primero inmediatamente posterior al flash, y 41,475 ms y 42,894 ms los siguientes. Las salidas permanecieron apagadas; no se enviaron ON, V, IL, VL, RST ni SAVE.

3. Resultados

Tabla 3. Ensayo de la rama positiva.

Magnitud	Resultado del ensayo
Carga	22,2 Ω
Entrada	10,0 V; 0,634 A; 6,328 W
Salida medida (referencia)	10,0 V; 0,450 A
Salida indicada (pantalla)	10,0 V; 0,45 A
Rendimiento total	≈ 71 %
Rendimiento neto estimado	≈ 83 %

Tabla 4. Ensayo de la rama negativa.

Magnitud	Resultado del ensayo
Carga	22,0 Ω
Entrada	10,0 V; 0,650 A; 6,497 W
Salida medida (referencia)	10,0 V; 0,455 A
Salida indicada (pantalla)	10,0 V; 0,45 A
Rendimiento total	≈ 70 %
Rendimiento neto estimado	≈ 81 %

Tabla 5. Margen de error observado luego de la calibración.

Rama	Error de tensión	Error de corriente
Positiva	≤ 0,10 V (≤ 0,8 %)	< 0,01 A (< 2,2 %)
Negativa	≤ 0,10 V (≤ 0,8 %)	< 0,005 A (< 1,1 %)

En los ensayos individuales se calculó Pout = V^2/R. La rama positiva entregó 4,50 W sobre 22,2 Ω: 71 % total y 83 % neto estimado. La negativa entregó 4,55 W sobre 22,0 Ω: 70 % y 81 %. La estimación neta descuenta el reposo informado de 0,087 A a 10 V (0,87 W); no representa la eficiencia aislada de una etapa. OC y OV apagaron y enclavaron el PWM; RST borró la falla sin reactivar la potencia. Con Ki = 3,50, el USB mostró ambas ramas cerca de ±5,00 V, Vin de 9,94 V a 10,02 V, VALP = VALN = 1, edades de 1 ms a 8 ms y sin fallas. Antes de la arquitectura final se completaron tres GET por consola y tres por web. La versión con cola central unificada añadió tres GET web físicos con EP = EN = 0, fallas OC/OV en 0, validez y disponibilidad en 1 y ciclos útiles en 0 %. No se repitieron carga, encendido ni protecciones.



Figura 5. Prototipo utilizado durante la verificación funcional.

Tabla 6. Ensayo adicional de ambas ramas durante la EGB.

Magnitud	Rama positiva	Rama negativa
Carga	22,2 Ω	22,0 Ω
Tensión de salida	5,00 V	5,00 V
Corriente indicada	0,22 A	0,22 A
Entrada total	10,00 V; 0,427 A; 4,275 W	
Potencia total calculada	2,26 W	
Rendimiento total	≈ 53 %	
Rendimiento neto estimado	≈ 66 %	

La potencia conjunta se calculó como $5^2/22{,}2 + 5^2/22{,}0 = 2{,}26$ W. Con 4,275 W de entrada, el rendimiento total fue 53 %; al descontar los 0,87 W de reposo informados, el neto estimado fue 66 %. Este punto de la EGB no reemplaza los ensayos individuales y mantiene la misma limitación de alcance del consumo de reposo.



Figura 6. Ensayo EGB con ambas ramas habilitadas a 5,00 V sobre cargas cementadas cercanas a 22 Ω; la fuente de entrada indica 10,00 V, 0,427 A y 4,275 W.

4. Discusión

La arquitectura conserva las decisiones críticas dentro de la Pico 2 W. Un retraso del navegador, del proceso web o del planificador Linux no interrumpe las tareas periódicas ni sustituye la protección local. Linux funciona como capa de acceso y supervisión. El dispositivo de carácter simplifica las aplicaciones porque oculta el detalle de la UART y ofrece una interfaz de archivo convencional [3], [4]. Como el controlador no interpreta comandos, la divergencia P/N frente a 0/1 se resolvió en cada cliente, manteniendo estable la interfaz presentada al usuario.

Los rendimientos totales fueron aproximadamente 71 % y 70 % con una rama a 10 V, y 53 % con ambas ramas a 5 V. Al descontar el reposo informado se estimaron 83 %, 81 % y 66 %. La primera serie relaciona la salida con toda la potencia de entrada medida; la segunda separa de forma aproximada 0,87 W sin atribuirlos a un subsistema. Los resultados corresponden a puntos concretos y no caracterizan todo el rango de 0 a 30 V. El diagnóstico USB de 1 Hz confirma continuidad y estado, pero no demuestra la cadencia exacta ni descarta sobreimpulsos entre muestras. Las protecciones digitales no sustituyen un comparador analógico para pulsos de pocos microsegundos. Tampoco se realizó un ensayo térmico prolongado a potencia máxima ni una evaluación de compatibilidad electromagnética.

El acceso se serializa por operación; por ello la consola y el panel pueden coexistir, pero el controlador no está diseñado para atender simultáneamente múltiples transacciones independientes. Esta restricción mantiene el protocolo simple y evita asociar una respuesta con el proceso equivocado, a costa de limitar el paralelismo.

5. Conclusiones

Se obtuvo una fuente partida funcional con regulación independiente, medición local, límites configurables y protecciones enclavadas. Diez tareas FreeRTOS separan adquisición, protección, cola

central tipada, colas de IRQ UART/encoder, conversiones I2C, controles, interfaz local, pantalla, guardado y comunicación. La EGB añadió controlador Linux, consola y web sin alterar el lazo crítico. Los clientes tradujeron P/N al protocolo final 0/1, superaron sus pruebas y se desplegaron en la Zero 2 W. Las seis lecturas GET iniciales correspondieron a arquitecturas anteriores. En la versión final se verificaron compilación, enlace, flash y USB, más tres GET web físicos completos: 590,525 ms el primero inmediatamente posterior al flash y 41,475 ms y 42,894 ms los siguientes. Se mantuvieron las salidas apagadas y no se repitieron carga ni protecciones. Los ensayos registraron errores inferiores a 0,10 V y 10 mA y operación simultánea a 5,00 V por rama, con rendimiento total cercano al 53 % y neto estimado del 66 % al descontar el reposo informado. Con Ki = 3,50 ambas mediciones permanecieron válidas durante la observación.

Como trabajo futuro se propone ampliar la caracterización en todo el rango, realizar ensayos térmicos prolongados, medir la respuesta transitoria y evaluar una protección analógica complementaria para eventos más rápidos que la cadena de conversión y software.

Financiamiento: el proyecto fue financiado íntegramente por el autor, quien adquirió los componentes, materiales e instrumentos utilizados.

Disponibilidad: el código fuente, los archivos de diseño y la documentación están disponibles en el repositorio académico de la asignatura.

6. Referencias

[1] AMAZON WEB SERVICES. FreeRTOS: RTOS fundamentals [en línea]. Disponible en: https://www.freertos.org/Documentation/01-FreeRTOS-quick-start/01-Beginners-guide/01-RTOS-fundamentals [consulta: 4 octubre 2026].

[2] RASPBERRY PI LTD. RP2350 datasheet [en línea]. Disponible en: https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf [consulta: 4 octubre 2026].

[3] THE LINUX KERNEL ORGANIZATION. The Linux driver model [en línea]. Disponible en: https://docs.kernel.org/driver-api/driver-model/index.html [consulta: 4 octubre 2026].

[4] RASPBERRY PI LTD. Configuration: device trees and UARTs [en línea]. Disponible en: https://www.raspberrypi.com/documentation/computers/configuration.html [consulta: 4 octubre 2026].

[5] TEXAS INSTRUMENTS. ADS1115: 16-bit, 860-SPS, 4-channel delta-sigma ADC with PGA, oscillator, voltage reference, comparator, and I2C interface. Datasheet SBAS444E, 2024.

[6] MICROCHIP TECHNOLOGY INC. TC4426/TC4427/TC4428: 1.5A dual high-speed power MOSFET drivers. Datasheet DS20001422, 2014.

[7] RASPBERRY PI LTD. Raspberry Pi Pico 2 datasheet [en línea]. Disponible en: https://datasheets.raspberrypi.com/pico/pico-2-datasheet.pdf [consulta: 4 octubre 2026].